OASIS VM 보안 아키텍처 분석

개요

OASIS VM은 Intel TDX(Trust Domain Extensions) 기반의 기밀 컴퓨팅 환경을 위해 설계된 보안 중심의 가상 머신 이미지입니다. 다층 보안 구조와 측정 가능한 부팅 프로세스를 통해 높은 수준의 보안성과 무결성을 제공합니다.

---

1. 보안 아키텍처 개요

신뢰 체인 (Chain of Trust)

```
Intel TDX → OVMF → Stage 1 → dm-verity → Stage 2 → Application
```

핵심 보안 원칙

- **측정된 부팅 (Measured Boot)**: 모든 단계가 측정되고 검증됨
- **무결성 보장**: dm-verity를 통한 파일시스템 무결성
- **격리**: 읽기 전용 루트와 임시 스토리지 분리
- **최소 권한**: 필요한 최소한의 권한만 부여

---

2. Intel TDX 기반 보안

TDX 개념

- **Trust Domain (TD)**: 격리된 실행 환경
- **RTMR (Runtime Measurement Register)**: 측정값 저장소
- **Remote Attestation**: 원격 증명 지원

RTMR 측정 지점

```
RTMR0: OVMF 펌웨어 측정
RTMR1: Stage 1 커널 및 initramfs 측정
RTMR2: 커널 명령줄 측정 (Stage 2 설정 포함)
RTMR3: Stage 2 루트 해시 측정
```

TDX 보안 기능

- **메모리 암호화**: 모든 TD 메모리 자동 암호화
- **DMA 보호**: 직접 메모리 접근 차단
- **하이퍼바이저 격리**: 하이퍼바이저로부터 보호

---

3. OVMF 펌웨어 보안

TDX 지원 OVMF

```
# TDX 특화 빌드
${S}/OvmfPkg/build.sh -p OvmfPkg/IntelTdx/IntelTdxX64.dsc
```

보안 부팅 지원 (선택적)

```
OVMF_SECURE_BOOT_FLAGS = "-DSECURE_BOOT_ENABLE=TRUE ${OVMF_SECURE_BOOT_EXTRA_F
```

보안 부팅 구성 요소

- **Platform Key (PK)**: 최상위 키
- **Key Exchange Key (KEK)**: 키 교환용
- **Signature Database (db)**: 허용된 서명
- **Forbidden Signature Database (dbx)**: 금지된 서명

펌웨어 측정

- OVMF 코드와 설정이 RTMR0에 측정됨
- 부팅 과정의 모든 변경사항 추적

---

4. Stage 1 보안 메커니즘

커널 보안 기능

```
KERNEL_FEATURES += "tdx.scc security-mitigations.scc disk-encryption.scc"
```

보안 완화 기법 (security-mitigations.scc)

- **KASLR**: 커널 주소 공간 레이아웃 랜덤화
- **SMEP/SMAP**: 실행 방지 및 접근 방지
- **Control Flow Integrity**: 제어 흐름 무결성
- **Stack Canaries**: 스택 오버플로우 방지

커널 명령줄 보안

```
# 측정된 설정 매개변수
oasis.stage2.roothash=<HASH> # dm-verity 루트 해시
oasis.stage2.hash_offset=<OFFSET> # 해시 오프셋
oasis.stage2.storage_mode=<MODE> # 스토리지 모드
```

보안 검증

```
# 필수 매개변수 검증
if [[ -z "${stage2_root_hash}" || -z "${stage2_hash_offset}" ]]; then
echo "Missing stage 2 configuration."
exit 1
fi
```

dm-verity 무결성 검증

```
veritysetup open \
--hash-offset=${stage2_hash_offset} \
--panic-on-corruption \ # 손상 시 패닉
/dev/mapper/part-rootfs \
stage2 \
/dev/mapper/part-rootfs \
```

```
${stage2_root_hash}
```

dm-verity 보안 특징

- **해시 트리**: 블록 단위 무결성 검증
- **루트 해시**: 전체 파일시스템의 무결성 요약
- **읽기 전용**: 검증된 데이터만 접근 가능
- **실시간 검증**: 블록 읽기 시마다 검증

---

5. 스토리지 보안

파티션 격리

```
# dm-linear을 통한 논리적 파티션 분리
echo "0 ${stage2_storage_offset} linear /dev/vda 0" | dmsetup create part-roo
echo "0 ${stage2_storage_size} linear /dev/vda ${stage2_storage_offset}" | dmse
```

임시 암호화 (Ephemeral Encryption)

디스크 모드 암호화

```
# 임시 키 생성 (재부팅 시 소실)
storage_key=$(dd if=/dev/urandom bs=1024 count=1 2>/dev/null | tr -dc 'a-zA
```

```
# LUKS2 + dm-integrity 설정
```

```
echo -n "${storage_key}" | cryptsetup luksFormat \
--type luks2 \
--integrity hmac-sha256 \ # 무결성 검증
--disable-locks \
--header /tmp/storage-luks2-header \
/dev/mapper/part-storage
```

보안 특징

- **임시성**: 재부팅 시 키 소실로 데이터 접근 불가
- **무결성**: HMAC-SHA256을 통한 데이터 무결성
- **메모리 저장**: LUKS 헤더를 tmpfs에 저장

RAM 모드 보안

```
# 메모리 기반 임시 스토리지
mount none -t tmpfs -o size=${stage2_storage_size} /stage2/storage
```

---

6. Stage 2 보안 구조

읽기 전용 루트 파일시스템

- **squashfs**: 압축된 읽기 전용 파일시스템
- **무결성**: dm-verity로 보호됨
- **변조 방지**: 런타임 수정 불가능

파일시스템 격리

```
/stage2/ # 읽기 전용 (squashfs + dm-verity)
├── bin/ # 실행 파일
├── lib/ # 라이브러리
├── etc/ # 설정 파일 (읽기 전용)
├── storage/ # 읽기-쓰기 (암호화된 임시 스토리지)
├── var -> /storage/var # 변수 데이터 (바인드 마운트)
├── tmp/ # 임시 파일 (tmpfs)
└── run/ # 런타임 데이터 (tmpfs)
```

네트워크 보안

```
# 최소한의 네트워크 설정
ip link set lo up # 루프백만 활성화
udhcpc -i eth0 -q -n # DHCP 클라이언트 (필요시)
```

---

7. 컨테이너 보안 (Podman 버전)

컨테이너 격리

```
# cgroups v2 마운트 (리소스 제한)
mount -t cgroup2 none /sys/fs/cgroup
```

```
# 네임스페이스 격리
mkdir -p /dev/shm
mount -t tmpfs none /dev/shm
```

Podman 보안 기능

- **Rootless 실행**: 루트 권한 없이 컨테이너 실행
- **SELinux 지원**: 강제 접근 제어 (선택적)
- **네임스페이스 격리**: PID, 네트워크, 마운트 격리
- **cgroups 제한**: CPU, 메모리, I/O 제한

컨테이너 스토리지 보안

```
# 컨테이너 데이터 격리
mkdir -p /storage/containers/{run,graph,graph/tmp,net}

# 권한 설정
chmod 0666 /dev/net/tun # TUN 디바이스 접근
```

---

8. 런타임 보안 모니터링

시스템 무결성 검증

```
# Podman 시스템 정리 및 검증
podman system migrate
podman system prune --external
```

로그 및 감사

- **커널 로그**: dmesg를 통한 시스템 이벤트
- **dm-verity 로그**: 무결성 검증 실패 기록
- **컨테이너 로그**: Podman 이벤트 추적

---

9. 공격 벡터 및 완화

잠재적 공격 벡터

1. 부팅 과정 공격

- **완화**: 측정된 부팅과 dm-verity 검증
- **탐지**: RTMR 값 변경으로 탐지 가능

2. 런타임 메모리 공격

- **완화**: TDX 메모리 암호화
- **격리**: 하이퍼바이저로부터 보호

3. 스토리지 공격

- **완화**: 임시 암호화와 무결성 검증
- **제한**: 읽기 전용 루트 파일시스템

4. 네트워크 공격

- **완화**: 최소한의 네트워크 노출
- **격리**: 컨테이너 네트워크 네임스페이스

5. 사이드 채널 공격

- **완화**: TDX 하드웨어 보호
- **완화**: 보안 완화 기법 적용

보안 모니터링 지점

```
# dm-verity 무결성 실패 시 패닉
--panic-on-corruption

# 커널 보안 이벤트 로깅
# 컨테이너 런타임 이벤트 추적
```

---

10. 원격 증명 (Remote Attestation)

TDX 증명 프로세스

1. **Quote 생성**: TD 내부에서 측정값 포함한 Quote 생성
2. **검증**: 원격 검증자가 Quote 검증
3. **신뢰 설정**: 검증 성공 시 신뢰 관계 설정

측정값 구성

```
RTMR0: OVMF 펌웨어 해시

RTMR1: Stage 1 커널 + initramfs 해시

RTMR2: 커널 명령줄 해시 (설정 포함)

RTMR3: Stage 2 루트 해시
```

증명 데이터 활용

- **설정 검증**: 예상된 설정으로 부팅되었는지 확인
- **무결성 확인**: 시스템이 변조되지 않았는지 검증
- **신뢰 결정**: 애플리케이션 데이터 전송 여부 결정

---

11. 보안 설정 권장사항

운영 환경 보안

```
# 1. 강력한 루트 해시 사용

oasis.stage2.roothash=<STRONG_HASH>



# 2. 적절한 스토리지 모드 선택

oasis.stage2.storage_mode=disk  # 또는 ram



# 3. 최소한의 네트워크 노출

# 필요한 포트만 개방
```

개발 환경 보안

```
# 1. 재현 가능한 빌드 환경
make clean && make  # 깨끗한 빌드

# 2. 서명된 이미지 사용
# 프로덕션에서는 서명된 Stage 2 이미지 사용

# 3. 정기적인 보안 업데이트
# 커널 및 패키지 업데이트
```

모니터링 및 로깅

```
# 1. 시스템 로그 수집
journalctl -f

# 2. dm-verity 이벤트 모니터링
dmesg | grep verity

# 3. 컨테이너 이벤트 추적
podman events
```

---

12. 보안 한계 및 고려사항

현재 한계

- **키 관리**: 임시 키로 인한 데이터 지속성 부족
- **네트워크 보안**: 기본적인 네트워크 보안만 제공
- **로깅**: 제한적인 보안 이벤트 로깅

향후 개선 방향

- **영구 스토리지**: 키 관리 시스템 통합
- **네트워크 보안**: TLS/mTLS 기본 지원
- **감사 로깅**: 포괄적인 보안 이벤트 로깅
- **정책 엔진**: 세밀한 보안 정책 제어

운영 고려사항

- **성능 영향**: dm-verity 검증으로 인한 I/O 오버헤드
- **메모리 사용**: 해시 트리 저장을 위한 추가 메모리
- **복잡성**: 다층 보안 구조로 인한 디버깅 복잡성

이 보안 아키텍처는 기밀 컴퓨팅 환경에서 높은 수준의 보안성을 제공하며, Intel TDX의 하드웨어 보안 기능과 소프트웨어 보안 메커니즘을 효과적으로 결합합니다.